

2. CÓDIGOS

(edición 2010)

Transmisión y codificación de la información

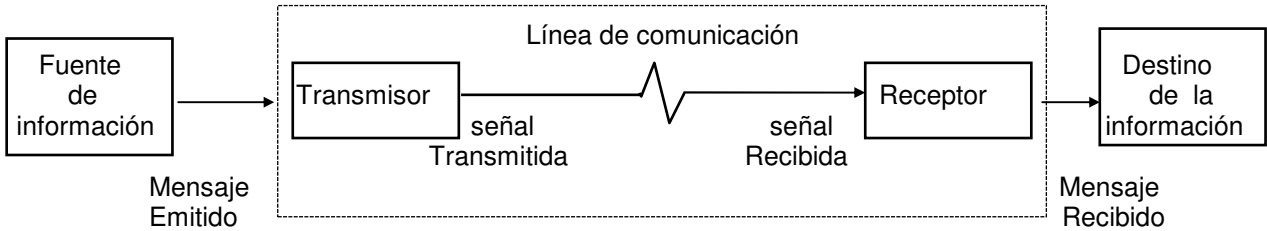


Figura 2-1.

Definiciones:

Mensaje: Cualquier dato sobre algún suceso ocurrido.

Canales de comunicación: Enlaces informativos que sirven para transmitir mensajes.

Señales: Portadores físicos de la información en los canales de información.

Codificación: Conversión de una señal de una forma a otra.

Por ejemplo en la transmisión de un telegrama. El mensaje se escribe en un formulario telegráfico que se convierte a la forma del código Morse que consta de puntos y rayas transmitidos mediante impulsos de corriente, largos y cortos por la línea de comunicación. En la recepción, por los impulsos captados se **regenera** el texto inicial del mensaje.

Se observa la presencia del mensaje en calidad de **señales** de diferentes formas:

- de texto literal
- puntos y rayas del código Morse
- impulsos de corriente en la línea de comunicación

De una forma a la siguiente, hay una codificación.

Alfabeto: Grupo de símbolos (letras, palabras, puntos, rayas, etcétera) que tengan un sentido conocido tanto por el remitente como por el destinatario del mensaje. Los propios símbolos se determinan por convenio de las partes.

Para la transmisión, el remitente elige del alfabeto, un símbolo tras otro, los convierte en las señales correspondientes y los transmite por el canal de comunicación. En éste, las señales se someten a la influencia de las interferencias, lo que provoca su distorsión. Así, las señales en la parte de recepción van a diferir de las señales enviadas al canal de comunicación.

El proceso de recepción consiste en que el destinatario al recibir cualquier señal debe identificarla con uno de los símbolos existentes del alfabeto, excluyendo los demás. Si hay grandes distorsiones en el canal, esta tarea de decodificación puede presentar grandes dificultades. Los métodos para superar estas dificultades constituyen la esencia de la Teoría de la Comunicación.

En los sistemas técnicos se emplean alfabetos de varios tipos. Pero por diversos motivos tiene gran empleo el alfabeto Binario, que sólo utiliza dos tipos de símbolos asignados convencionalmente por 0 y 1. Mediante el alfabeto binario un mensaje está constituido por una secuencia de ceros y unos, como por ejemplo en 1001001. El número total de mensajes que constan de **m** letras del alfabeto binario es igual a 2^m .

Códigos binarios.

Toda información de tipo discreta puede ser directamente codificada en binario.

Los códigos binarios se generan en función de las necesidades. Así, si por ejemplo tenemos que discernir entre 4 objetos O_1, O_2, O_3, O_4 y podemos asignarles las combinaciones de 2 dígitos binarios 00, 01, 10 y 11 respectivamente.

En general, con **N** cifras binarias se pueden obtener 2^N combinaciones. Cada una de estas se puede asignar a un objeto distinto. Por ello el número total de asignaciones es el de permutaciones de las 2^N combinaciones, vale decir $2^N!$, que constituyen otros tantos códigos binarios.

Si $N=1$, $2^1 = 2$ $1 \times 2 = 2 = 2!$

Si $N=2$, $2^2 = 4$ $1 \times 2 \times 3 \times 4 = 24 = 4!$

En éste último, entonces, para representar 4 objetos podemos escoger entre 24 códigos de dos dígitos binarios.

Ejemplo: Asignación de códigos para cuatro colores

Código	asignaciones posibles de colores
0 0	A A A A A R R R R R V V V V V N N N N N A = Azul
0 1	R R V V N N A A V V N N A A R R N N A A R R V V R = Rojo
1 0	V N R N R V V N A N A V R N A N A R R V A V A R V = Verde
1 1	N V N R V R N V N A V A N R N A R A V R V A R A N = Negro

Tabla 2-1.

Códigos binarios específicos.

Estudiaremos a continuación aquellos cuyo uso es más frecuente debido a que poseen alguna propiedad particular.

Códigos binarios continuos y cíclicos.

Combinaciones binarias adyacentes: son aquellas que difieren en un solo bit.
Un código binario es *continuo* si las combinaciones correspondientes a números decimales consecutivos son adyacentes.
Además, un código binario *continuo es cíclico* si la última combinación es adyacente a la primera.

Código reflejado de Gray.

A partir del código binario natural de dos bits, crearemos el código de Gray de dos dígitos. Y reflejando sucesivamente los resultados se generará el código para **n** cantidad de dígitos.

1) Código natural de dos bits

0 0	0
0 1	1
1 0	2
1 1	3

2) Asignamos la combinación 10 al 3 y 11 al 2 para obtener el código continuo y cíclico de dos bits.

0 0	0
0 1	1
1 1	2
1 0	3

3) Reflejemos el código anterior

0 0	←
0 1	←
1 1	←
1 0	←

1 0	←
1 1	←
0 1	←
0 0	←

4) Para lograr el reflejado de tres bits basta agregar una tercera columna a la izquierda con ceros para las primeras 4 filas y con unos para las 4 filas restantes.

0 0 0	0
0 0 1	1
0 1 1	2
0 1 0	3

1 1 0	4
1 1 1	5
1 0 1	6
1 0 0	7

5) Tomando el de tres bits, reflejándolo y agregando la columna siguiente con la mitad superior de las filas conteniendo ceros y la mitad inferior unos, se logra el reflejado de cuatro bits.

Gray	dec.
0 0 0 0	0
0 0 0 1	1
0 0 1 1	2
0 0 1 0	3
0 1 1 0	4
0 1 1 1	5
0 1 0 1	6
0 1 0 0	7

1 1 0 0	8
1 1 0 1	9
1 1 1 1	10
1 1 1 0	11
1 0 1 0	12
1 0 1 1	13
1 0 0 1	14
1 0 0 0	15

El código Gray se emplea principalmente para la conversión de información analógica a digital.
Una gran ventaja de este código es su facilidad de conversión al código binario natural (que veremos en una unidad subsiguiente).

Tabla 2-3. Código Reflejado de Gray

Código progresivo Johnson.

Es otro código continuo y cíclico. Su capacidad de codificación es **2.n**, siendo **n** el número de posiciones binarias. Es muy práctico para sistemas electrónicos de conteo.
Así para n=5, 2x5=10, lo que tan solo permite codificar los diez símbolos decimales.

Dec.	Johnson
0	0 0 0 0 0
1	0 0 0 0 1
2	0 0 0 1 1
3	0 0 1 1 1
4	0 1 1 1 1
5	1 1 1 1 1
6	1 1 1 1 0
7	1 1 1 0 0
8	1 1 0 0 0
9	1 0 0 0 0

Tabla 2-4. Código Johnson

Códigos decimales codificados en binario (BCD).

En éstos, cada cifra decimal se codifica directamente en un código binario. Para representar los diez símbolos de una cifra, se requieren cuatro bits, porque con tres, sólo podríamos codificar $2^3 = 8$ símbolos. Por lo tanto, de las $2^4 = 16$ combinaciones de cuatro bits, sólo se ocupan diez.
Ésta sobreabundancia hace que se restrinja el empleo de éstos códigos BCD al diseño de sistemas de control digitales y calculadoras digitales de pequeña capacidad, como las de bolsillo.

Códigos BCD ponderados.

Son aquellos en los que a cada posición o cifra binaria se le asigna un peso y por lo tanto, el número decimal equivalente surge de sumar los pesos de las posiciones que posean el valor 1.
De los BCD ponderados, los más importantes son el BCD natural (1 2 4 8), el BCD Aiken (1 2 4 2) y el BCD 1 2 4 5.

Por otra parte, el código Aiken, además de ser BCD ponderado es **autocomplementario**: la combinación correspondiente al complemento a nueve de n , es decir $9 - n$, se obtiene invirtiendo la combinación correspondiente a n , es decir cambiando los ceros por unos y viceversa. Ej.: si $n = 4$, entonces tenemos que $9 - 4 = 5$, luego como la combinación para 4 es 0 1 0 0, su complemento es 1 0 1 1 que corresponde a la combinación para 5.

DÍGITO DECIMAL	BCD NATURAL	BCD AIKEN	
	8 4 2 1	2 4 2 1	5 4 2 1
0	0 0 0 0	0 0 0 0	0 0 0 0
1	0 0 0 1	0 0 0 1	0 0 0 1
2	0 0 1 0	0 0 1 0	0 0 1 0
3	0 0 1 1	0 0 1 1	0 0 1 1
4	0 1 0 0	0 1 0 0	0 1 0 0
5	0 1 0 1	1 0 1 1	1 0 0 0
6	0 1 1 0	1 1 0 0	1 0 0 1
7	0 1 1 1	1 1 0 1	1 0 1 0
8	1 0 0 0	1 1 1 0	1 0 1 1
9	1 0 0 1	1 1 1 1	1 1 0 0

Tabla 2-5 : códigos BCD ponderados

Códigos BCD no-ponderados.

No tienen asignación de peso. De éstos el más utilizado es el denominado **exceso de 3**. Es autocomplementario como el Aiken y se obtiene adicionando 3 a cada combinación del BCD natural.

Dígito Decimal	B C D ex-3
0	0 0 1 1
1	0 1 0 0
2	0 1 0 1
3	0 1 1 0
4	0 1 1 1
5	1 0 0 0
6	1 0 0 1
7	1 0 1 0
8	1 0 1 1
9	1 1 0 0

Tabla 2-6.
Codigo BCD ex-3

La codificación de una cantidad decimal de dos o más cifras mediante un código BCD se realiza cifra por cifra. Así por ejemplo, la cantidad decimal 748 se representa:

B C D	7	4	8
Natural	0 1 1 1	0 1 0 0	1 0 0 0
Exc. 3	1 0 1 0	0 1 1 1	1 0 1 1
Aiken	1 1 0 1	0 1 0 0	1 1 1 0

Ejemplo 2-1.

Códigos alfanuméricos.

Un código alfanumérico es un código binario que permite representar los 10 símbolos decimales y los 26 símbolos alfabéticos (letras) más un cierto número de símbolos especiales, tales como: ; , \$ " etcétera.

El mínimo número de bits para codificar es seis, ya que $2^5 = 32$ no es suficiente.

Los códigos alfanuméricos más comunes son el **ASCII** (**A**merican **S**tandard **C**ode for **I**nterchange of **I**nformation) que puede ser de 7 u 8 dígitos, y el **EBCDIC** (**E**xtended **B**inary **C**oded **D**ecimal **I**nterchange **C**ode) que tiene 8 dígitos. Existen otros como el interno de máquina y el código Hollerich para tarjeta perforada que ya perdió casi totalmente su utilidad.

Un ejemplo de codificación en el código interno "6 bits":

J	U	A	N
100001	110100	010001	100101

Códigos Detectores de Errores.

En el manejo, y especialmente en la transmisión de una información numérica, es posible que se produzcan errores debido a la presencia de ruido en el proceso o por avería de alguno de los componentes.

Cuando en un código binario se utilizan todas las combinaciones posibles (2^n) de sus n posiciones, es imposible la detección de un error, porque una combinación del código se transformará en otra que también pertenece a él. Por consiguiente la detección de errores en un código binario se logra no utilizando todas las combinaciones posibles. Pero esta condición, aunque es necesaria no es suficiente para que el código permita detectar errores. Por ejemplo, el código BCD ex 3 no utiliza más que diez combinaciones de las dieciséis posibles de los cuatro bits, pero, si por un error en un bit, la combinación 0011 se convierte en 0111, no es posible detectarlo, porque ambas combinaciones pertenecen al código.

Para establecer la condición necesaria y suficiente para que un código binario permita detectar errores, definiremos el concepto de distancia mínima de un código.

Distancia mínima de un código.

En un código se define a la distancia mínima, como la menor de las distancias entre dos combinaciones binarias pertenecientes al mismo. El valor de la distancia mínima de los códigos estudiados hasta ahora, es la unidad, y, por lo tanto, un error en uno solo de los bits de una combinación binaria perteneciente a cualquiera de ellos, puede convertirlo en otra combinación valida para dicho código, haciendo que el error no sea detectable.

De todo lo dicho se deduce que, **para que un código pueda detectar errores, su distancia mínima ha de ser superior a la unidad.**

Existen diversos tipos de códigos detectores de errores, entre los cuales se encuentran los códigos de control de paridad y los códigos de peso constante (entendiendo por peso de una combinación binaria al número de unos lógicos de la misma).

Códigos de control de paridad.

Los códigos de control de paridad se obtienen añadiendo a las combinaciones de los códigos de distancia unidad anteriormente estudiados, un bit llamado de paridad. Si el código que se desea obtener es de paridad par, dicho bit será tal que el número de unos en cada combinación del nuevo código se par. Si, por el contrario, se desea un código de paridad impar, el bit añadido a cada combinación ha de ser tal que la combinación resultante tenga un número impar de unos.

La detección de errores en estos códigos consiste en comprobar al recibir la información, si el número de unos de cada combinación es par (códigos de paridad par) o impar (códigos de paridad impar).

La principal ventaja de estos códigos es que se forman a partir de cualquier otro código o combinación binaria.

dígito decimal	BCD ex de 3	bit paridad impar	código resultante
0	0 0 1 1	1	0 0 1 1 1
1	0 1 0 0	0	0 1 0 0 0
2	0 1 0 1	1	0 1 0 1 1
3	0 1 1 0	1	0 1 1 0 1
4	0 1 1 1	0	0 1 1 1 0
5	1 0 0 0	0	1 0 0 0 0
6	1 0 0 1	1	1 0 0 1 1
7	1 0 1 0	1	1 0 1 0 1
8	1 0 1 1	0	1 0 1 1 0
9	1 1 0 0	1	1 1 0 0 1

Ejemplo 2-2. Cálculo del bit de paridad

Códigos de peso constante.

Entre los códigos de peso constante se encuentran el 2 entre 5 y el biquinario que es un código de 7 bits. Ambas combinaciones poseen solamente dos unos lógicos.

dígito decimal	código 2 entre 5	cód.biquinario 5 0 4 3 2 1 0
0	0 1 1 0 0	0 1 0 0 0 0 1
1	1 1 0 0 0	0 1 0 0 0 1 0
2	1 0 1 0 0	0 1 0 0 1 0 0
3	1 0 0 1 0	0 1 0 1 0 0 0
4	0 1 0 1 0	0 1 1 0 0 0 0
5	0 0 1 1 0	1 0 0 0 0 0 1
6	1 0 0 0 1	1 0 0 0 0 1 0
7	0 1 0 0 1	1 0 0 0 1 0 0
8	0 0 1 0 1	1 0 0 1 0 0 0
9	0 0 0 1 1	1 0 1 0 0 0 0

(pesos)

Tabla 2-7. Códigos de peso constante

Los códigos estudiados son de distancia mínima de dos y permiten, pues la detección de errores de un bit. Para poder detectar errores de más de un bit es necesario utilizar códigos de distancia mínima superior a dos. En general el número de bits erróneos que se pueden detectar es igual al número en que la distancia mínima es superior a la unidad.

Códigos Correctores de Errores.

Los códigos correctores de errores, no sólo indican la existencia de un error como los que hemos estudiado en el apartado anterior, sino que proporcionan información de cual es la cifra o cifras binarias erróneas y, por consiguiente permiten su corrección invirtiendo simplemente el bit correspondiente.

Estos códigos se utilizan cuando no es posible o se hace muy difícil pedir la retransmisión de una información frente a la detección de un error.

De producirse un error en un código detector de errores, la combinación errónea obtenida posee como mínimo dos adyacentes pertenecientes al código, y no es posible discernir de cual de estos procede. Por ejemplo, en el código BCD ex 3 con paridad impar la distancia mínima es dos, si se detecta la combinación errónea '10001' es imposible conocer si el error se ha producido en el primer bit y la combinación correcta es '10000', en el segundo y es correcto '10011', tercero y es '10101' o el cuarto y es '11001'.

Por lo tanto, para poder corregir errores, la distancia mínima del código ha de ser superior a dos. Si la distancia mínima de un código es tres, la combinación obtenida por error en un bit, es adyacente a una sola combinación del código y es posible conocer cuál es el bit erróneo. Así, un código de distancia mínima tres, permite detectar errores de 2 bits o corregir errores de 1 bit. En general, la distancia mínima de un código para que permita corregir errores de *n* bits ha de ser **dm = 2n + 1.**

Nos limitaremos a estudiar los códigos correctores de errores de un bit, cuya distancia mínima es tres y entre ellos los de mayor difusión, que son los códigos de HAMMING.

Código corrector de HAMMING.

Estos códigos están basados en la adición de un código de distancia unidad para **n** bits, de **p** bits, obteniéndose un nuevo código de **n + p** bits. En este nuevo código se realizan **p** detecciones de paridad en bits preseleccionados del mismo, obteniéndose un bit de paridad PAR. El conjunto de las ecuaciones de control de paridad forman un número del sistema binario que indica la posición del bit erróneo. En caso que no exista error, el número binario logrado debe ser igual a cero. El número de **p** bits añadidos ha de ser suficiente para permitir la detección de error en alguna de las **n + p** posiciones o la ausencia del mismo. Dado que con **p** bits se obtienen **2^p** combinaciones, se ha de cumplir la relación **2^p ≥ n + p + 1**.

Como ejemplo realizaremos el código de Hamming obtenido a partir del código BCDN. En este código **n = 4** y, por lo tanto el número de bits que se han de añadir es 3, dado que **2³ = 4 + 3 + 1**. Para detectar los siete posibles errores de un bit en cada una de las posiciones y la ausencia de error son necesarias ocho combinaciones binarias que denominaremos correctoras de error. Dichas combinaciones se obtienen mediante 3 bits, **c₃, c₂, c₁**, y el número decimal equivalente al binario formado por ellos ha de indicar la posición errónea.

Veremos ahora la forma de generar cada uno de los bits de la combinación correctora de errores. En la tabla, se presentan todas las combinaciones de los bits **c₃, c₂, c₁**.

numero decimal	c ₃	c ₂	c ₁
correcto 0	0	0	0
(b ₁) 1	0	0	1
(b ₂) 2	0	1	0
(b ₃) 3	0	1	1
(b ₄) 4	1	0	0
(b ₅) 5	1	0	1
(b ₆) 6	1	1	0
(b ₇) 7	1	1	1

Tabla 2-8. Asignación binaria para cada bit del código

El bit **c₁** ha de tomar el valor 1 si se produce un error en los bits **b₁, b₃, b₅, b₇**, de la combinación del código. Si el número de unos existentes en esas cuatro posiciones es siempre par, un error en uno cualquiera de esos cuatro bits lo convierte en impar. Por lo tanto **c₁** ha de valer 1 si el número de unos en las posiciones **b₁, b₃, b₅, b₇**, es impar y cero en caso contrario. Esto se expresa algebraicamente de la siguiente forma:

c₁ = b₁ ⊕ b₃ ⊕ b₅ ⊕ b₇

Donde ⊕ es el símbolo de la función reunión excluyente que se estudiará más adelante. De igual forma se deduce que **c₂** y **c₃** han de obtenerse por medio de las expresiones:

c₂ = b₂ ⊕ b₃ ⊕ b₆ ⊕ b₇
c₃ = b₄ ⊕ b₅ ⊕ b₆ ⊕ b₇

para lo cual ha de cumplirse la condición de que el número de unos ha de ser par en las combinaciones **b₂, b₃, b₆, b₇** y **b₄, b₅, b₆, b₇**.

Para lograr estas condiciones se han de generar adecuadamente los tres bits que se añaden a los cuatro de la combinación BCDN.

Dado que **b₁, b₂, b₄**, por ser potencias de 2 (tienen un solo 1 en su combinación binaria) aparecen en una sola expresión cada uno, los elegiremos como bits de control y lo añadimos a la combinación de entrada que será volcada en **b₃, b₅, b₆, b₇**. El bit **b₁** ha de valer uno si el número de unos de **b₃, b₅ y b₇**, es impar y cero en caso contrario, por lo tanto.

b₁ = b₃ ⊕ b₅ ⊕ b₇

De igual forma **b₂** y **b₄**; se han de obtener respectivamente:

b₂ = b₃ ⊕ b₆ ⊕ b₇
b₄ = b₅ ⊕ b₆ ⊕ b₇

De todo lo anterior se deduce el código de Hamming siguiente:

número decimal	b ₇	b ₆	b ₅	b ₄	b ₃	b ₂	b ₁
0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	0	1	1	0	0	1
3	0	0	1	1	1	1	0
4	0	1	0	1	0	1	0
5	0	1	0	1	1	0	1
6	0	1	1	0	0	1	1
7	0	1	1	0	1	0	0
8	1	0	0	1	0	1	1
9	1	0	0	1	1	0	0

Ejemplo 2-3.

Como ejemplo comprobaremos la detección de un error en el b_6 ; de la combinación 0011001 correspondiente al número decimal 2, la combinación errónea es 0111001.
Para detectarlo comprobaremos el valor lógico de c_3 , c_2 , c_1 .

$$c_3 = b_4 \oplus b_5 \oplus b_6 \oplus b_7 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$$
$$c_2 = b_2 \oplus b_3 \oplus b_6 \oplus b_7 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$
$$c_1 = b_1 \oplus b_3 \oplus b_5 \oplus b_7 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$$

En efecto, la combinación c_3 , c_2 , c_1 , es 110_2 , equivalente al número decimal 6.

Ejercicio:
Obtener un código corrector de Hamming a partir del código Johnson para los números del 0 al 9.

ANEXO I

Códigos de caracteres alfanuméricos.

Carácter	ASCII 7-8 bits	EBCDIC 8 bits		Carácter	ASCII 7-8 bits	EBCDIC 8 bits
A	0100 0001	1100 0001		0	0011 0000	1111 0000
B	0100 0010	1100 0010		1	0011 0001	1111 0001
C	0100 0011	1100 0011		2	0011 0010	1111 0010
D	0100 0100	1100 0100		3	0011 0011	1111 0011
E	0100 0101	1100 0101		4	0011 0100	1111 0100
F	0100 0110	1100 0110		5	0011 0101	1111 0101
G	0100 0111	1100 0111		6	0011 0110	1111 0110
H	0100 1000	1100 1000		7	0011 0111	1111 0111
I	0100 1001	1100 1001		8	0011 1000	1111 1000
J	0100 1010	1101 0001		9	0011 1001	1111 1001
K	0100 1011	1101 0010		espacio	0010 0000	0100 0000
L	0100 1100	1101 0011		.	0010 1000	0100 1011
M	0100 1101	1101 0100		+	0010 1011	0100 1110
N	0100 1110	1101 0101		\$	0010 0100	0101 1011
O	0100 1111	1101 0110		*	0010 1010	0101 1100
P	0101 0000	1101 0111		)	0010 1001	0101 1101
Q	0101 0001	1101 1000		-	0010 1101	0110 0000
R	0101 0010	1101 1001		/	0010 1111	0110 0001
S	0101 0011	1110 0010		,	0010 1100	0110 1011
T	0101 0100	1110 0011		=	0011 1101	0111 1110
U	0101 0101	1110 0100				
V	0101 0110	1110 0101				
W	0101 0111	1110 0110				
X	0101 1000	1110 0111				
Y	0101 1001	1110 1000				
Z	0101 1010	1110 1001				

Tabla 2A-1.